



Homework 2: Digital Clock Design

Name 1: Conor Murphy

Student No.: 23334046

Name 2: Gavin Mac Aonghusa

Student No.: 23378506

We declare that the work described in this report was done by us, and that the description and comments in this report are our own work, except where otherwise acknowledged. We have read the University's policy on plagiarism, and accept it.

Are you participating in the "Best Design" competition? Yes

If you choose "Yes", please give the detailed transistor number and specify the count basis in the report.

Signed: Gavin MacAonghusa

Date: 26 / 11 / 2024

Signed: Conor Murphy

Date: 26 / 11 / 2024

Maximum two students in a team. You also can do it in your own if you prefer. **One report is required for one team**, please stated clearly in the report who was responsible for which part of the design and how the co-design was conducted.

Contents

Contents.....	2
1. Introduction	2
2. Counters	2
3. Clock Mode	8
4. Alarm Mode	12
5. Stopwatch Mode.....	14
6. Timer Mode.....	15
7. Flash	17
8. Controller	18
9. User Interface	19
10. Number Of Transistors.....	20
11. Conclusions	21

1. Introduction

In this assignment the aim was to create a fully functioning digital clock and run it on Logism. In this report we will give the detailed design procedures that we executed in order to complete this task. The digital clock is required to carry out multiple functions. To display the time in both 24- and 12-hour mode, these were to be distinguishable by the means of an additional display of 'A' or 'P' for the 12 hour display. The clock must trigger an alarm at a set time by the user, this trigger should be displayed to the user for 30 seconds unless changed manually. You will see below that we opted to use LEDs to perform this display. The Clock must include a count up and count down timer, the count down timer must be able to be set by the user. We added a flashing LED display when the timer goes off similar to the design of the alarm.

In order to complete the above functions, we created many separate counters and logic circuits all combined under an accessible user interface. The report includes the designs of all sub-circuits in logism along with discussions and a detailed transistor count.

2. Counters

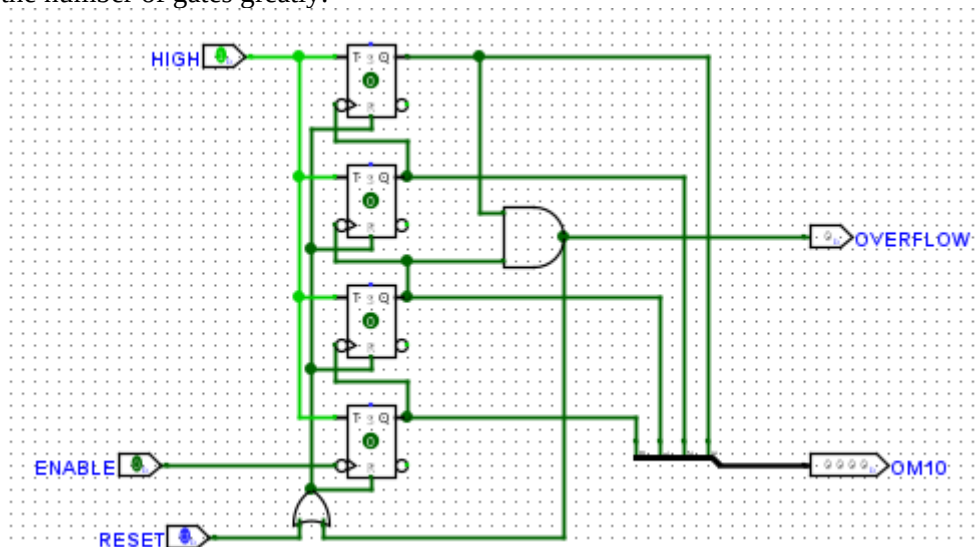
Counters were a key feature in our clock design, we used a combination of ripple counters and synchronous binary counters throughout. These counters are represented by subcircuits labelled MODULOs, due to the large amount needed in our design, we simplified these counters using Karnaugh maps and state tables for a reduction in the use of transistors.

Ripple Counters

Ripple Counters are series of flip flops in a chain, the output of one flip flop serves as the input for the next flip flop. These flip flops were essential for representing an increase of time. This is evident in our Clock mode.

MODULO 10

Modulo 10 represents the digits 0-9. In order to use this we needed four T flip-flops to represent the maximum binary output of 1001(9). Initially we used JK Flip-Flops but the use of T flip flops reduced the number of gates greatly.

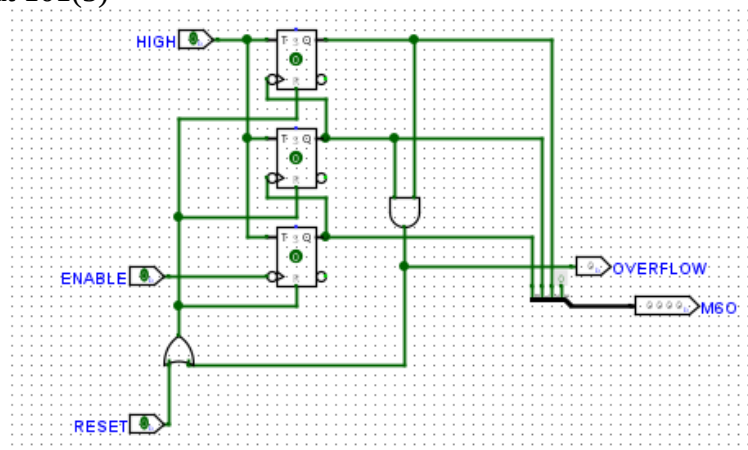


This ripple counter was easy to use as the output of each Flip-Flop triggered the one above it. In order to not surpass the output of (1001) we used an overflow component, this overflow triggered when T0 and T3 were both high, This reset all flip flops back to 0, so the output of the MODULO went from 9 to 0. It also acted as an input for other MODULOS in larger subcircuits, this will be discussed in other parts of the report.

For our clock we also needed a MODULO 6, 5, 4 and 3. These all worked in the same way as the MODULO 10, but used a smaller number of T flip-flops and overflowed at different outputs.

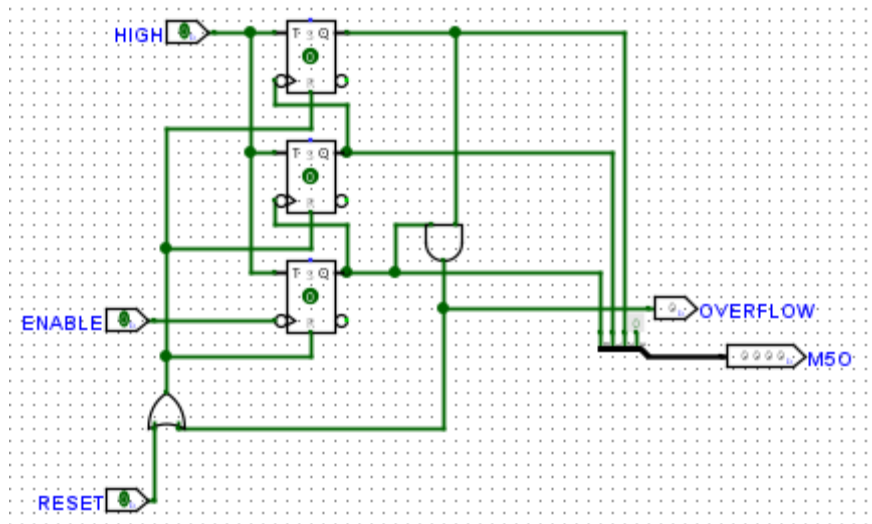
MODULO 6

The MODULO 6 represents the outputs 0-5, we needed 3 T flip-flops this needed to have an overflow at output 101(5)



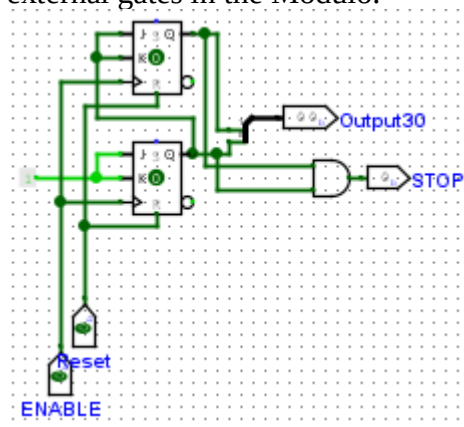
MODULO 5

The MODULO 5 represents the outputs 0-4, we needed 3 T flip-flops this needed to have an overflow at output 100(4)



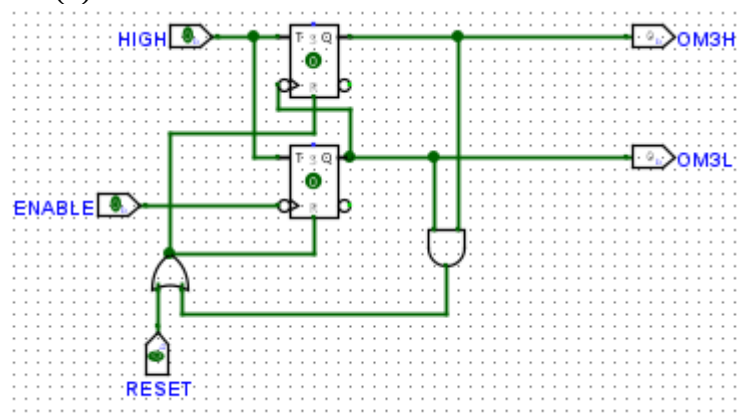
MODULO 4

The MODULO 4 represents the outputs 0-3, we needed 2 T flip-flops this needed to have an overflow at output 11(3). However in review we decided to opt for a synchronous counter for our MODULO 4 as it was not directly used to count clock cycles. This meant we used JK flip flops to reduce the number of external gates in the Modulo.



MODULO 3

The MODULO 3 represents the outputs 0-2, we needed 2 T flip-flops this needed to have an overflow at output 10(2)



For all of the T Flip-flops in the counters we used the falling edge setting. This was to ensure that there is no glitches when enabling the upper flip-flop. By using falling edge flip flops, the next flip flop toggles only after the current flip flop stabilizes.

Synchronous Counters

For our MODULO downs, that we use in the timer we opted to use JK synchronous counters, we used this form of counter as our clock input causes the Flip Flops to count down.

Synchronous counters have a common clock input for each flip flop. After analysing using both T flip flops and Jk Flip-flops we concluded that we would use JK flip-flops as less external gates were used in each MODULO.

MODULO 10 Down

Greater detailed design procedures were needed in order to simplify the synchronous counters and minimize gates used.

Our Modulo 10 is used to count down 9-0 we needed 4 JK Flip-Flops to execute this. The counter needed to have an overflow output that resets the Counter to 1001(9) and also act as an output for other MODULOS.

Design Procedure MODULO10Down State Table

Decimal	Binary	NEXT	J3	K3	J2	K2	J1	K1	J0	K0
9	1001	1000	X	0	0	X	0	X	X	1
8	1000	0111	X	1	1	X	1	X	1	X
7	0111	0110	0	X	X	0	X	0	X	1
6	0110	0101	0	X	X	0	X	1	1	X
5	0101	0100	0	X	X	0	0	X	X	1
4	0100	0011	0	X	X	1	1	X	1	X
3	0011	0010	0	X	0	X	X	0	X	1
2	0010	0001	0	X	0	X	X	1	1	X
1	0001	0000	0	X	0	X	0	X	X	1
0	0000	1001	1	X	0	X	0	X	1	X

Karnaugh Maps

J3	Q1Q0					J1	Q1Q0				
Q3Q2	00	01	11	10		Q3Q2	00	01	11	10	
00	1	0	0	0		00	0	0	x	x	
01	0	0	0	0		01	1	0	x	x	
11	x	x	x	x		11	x	x	x	x	
10	x	x	x	x		10	1	0	x	x	

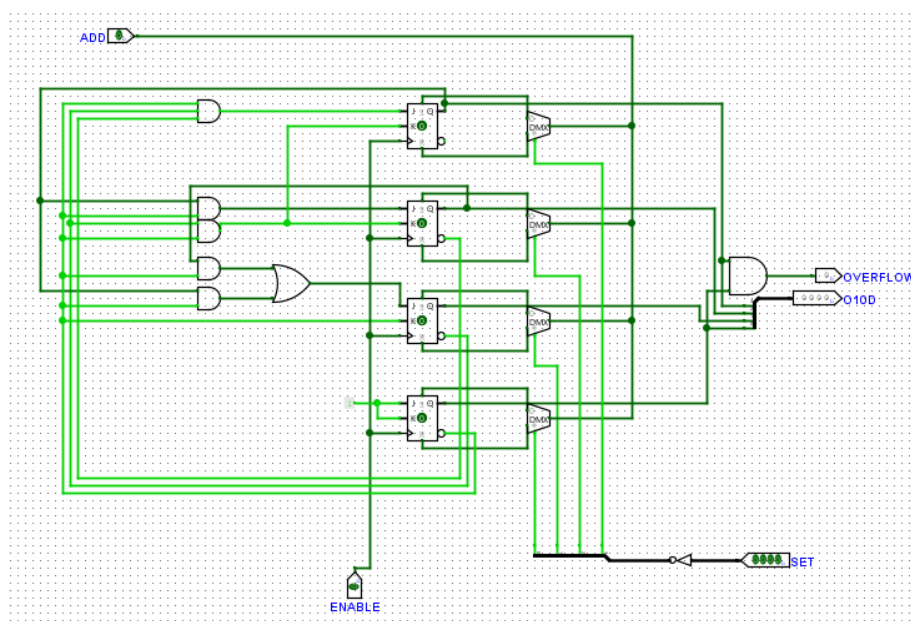
K3	Q1Q0					K1	Q1Q0				
Q3Q2	00	01	11	10		Q3Q2	00	01	11	10	
00	x	x	x	x		00	x	x		0	1
01	x	x	x	x		01	x	x		0	1
11	x	x	x	x		11	x	x	x	x	
10	1	0	x	x		10	x	x	x	x	

J2	Q1Q0					J0	Q1Q0				
Q3Q2	00	01	11	10		Q3Q2	00	01	11	10	
00	0	0	0	0		00	1	1	1	1	
01	x	x	x	x		01	1	1	1	1	
11	x	x	x	x		11	1	x	x	1	
10	1	0	x	x		10	1	1	x	x	

K2	Q1Q0					K0	Q1Q0				
Q3Q2	00	01	11	10		Q3Q2	00	01	11	10	
00	x	x	x	x		00	1	1	1	1	
01	1	0	0	0		01	1	1	1	1	
11	x	x	x	x		11	x	x	x	x	
10	x	x	x	x		10	1	1	x	x	

Flip Flop Outputs

J3=	Q2'Q1'Q0'
K3	Q1'Q0'
J2	Q3Q0
K2	Q1'Q0'
J1	Q3Q0'+Q2Q0'
K1	Q0'
J0	1
K0	1



In our circuit for the modulo, there are four demultiplexers included, this is essential for setting the timer and will be discussed in the timer section.

MODULO 6 Down

Our Modulo 6 is used to count down 5-0 we needed 3 JK Flip-Flops to execute this. The counter needed to have an overflow output that resets the Counter to 101(5) and also act as an output for other MODULOS.

State Table

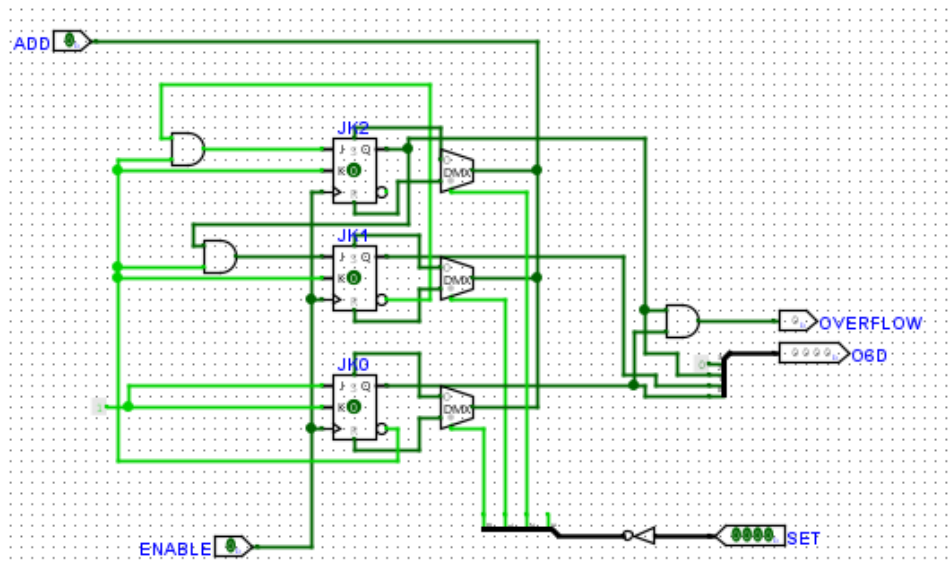
Present State	Q2	Q1	Q0	Next State	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
5	1	0	1	4	1	0	0	x	0	0	x	x	1
4	1	0	0	3	0	1	1	x	1	1	x	1	x
3	0	1	1	2	0	1	0	0	x	x	0	x	1
2	0	1	0	1	0	0	1	0	x	x	1	1	x
1	0	0	1	0	0	0	0	0	x	0	x	x	1
0	0	0	0	5	1	0	1	1	x	0	x	1	x

Karnaugh Map

J2	Q0		K2	Q0
Q2Q1	0 1		Q2Q1	0 1
00	1 0		00	x x
01	0 0		01	x x
11	x x		11	x x
10	x x		10	1 0
J1	Q0			
Q2Q1	0 1		K1	Q0
Q2Q1	0 1		Q2Q1	0 1
00	0 0		00	x x
01	x x		01	1 0
11	x x		11	x x
10	1 0		10	x x
J0	Q0			
Q2Q1	0 1		K0	Q0
Q2Q1	0 1		Q2Q1	0 1
00	1 x		00	x 1
01	1 x		01	x 1
11	x x		11	x x
10	1 x		10	x 1

Flip-Flop Output

J2=	Q1'Q0'
K2=	Q0'
J1=	Q2Q0'
K1=	Q0'
J0=	1
K0=	1



When creating and implementing all the counters, what we initially thought would be a reduction in gates by using JK flip flops in our Countdown and 4 MODULO. Turned out to have a increase in transistors as JK flip flops contain many more than T flip-flops. However this still simplified the external design, which lead to clarity and neatness of final internal elements of the clock.

3. Clock Mode

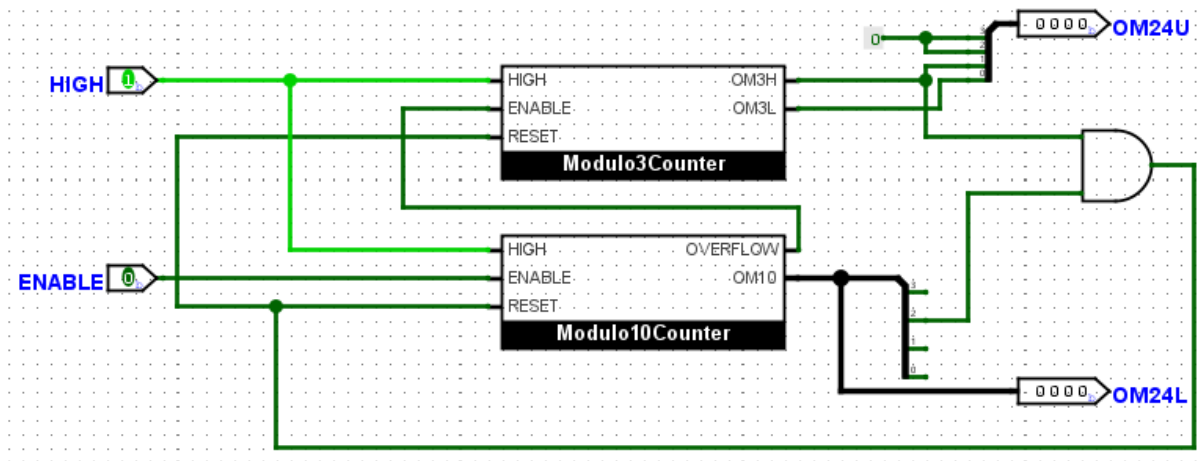
Counting up

The clock runs on a combination of the modulo 10 and modulo 6 counters when counting up through seconds and minutes. The clock input enables the modulo 10 seconds counter which in turn overflows to enable all above modulos. This is shown in the image below where we see that clock input enables the first modulo and the overflow enables the modulo of greater magnitude.

Through this structure we were able to implement the seconds and minutes of the clock which do not change between the 12 and 24 hour modes. However, for the hours section we needed two independent counters for 12 and 24 hours which would count up synchronously.

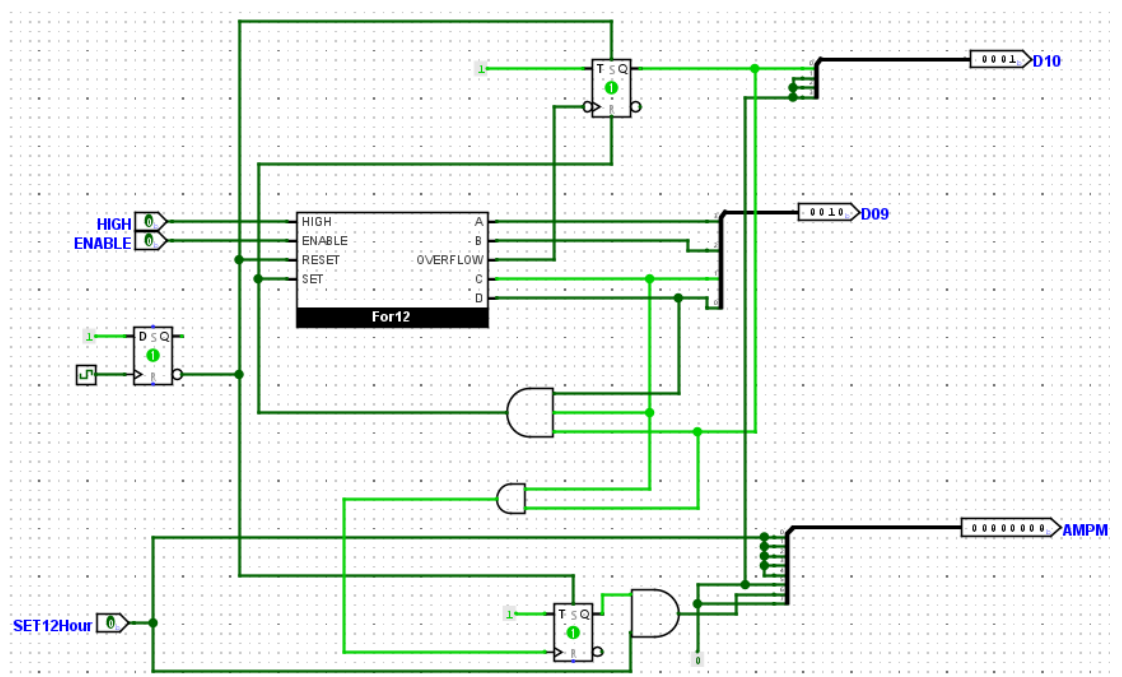
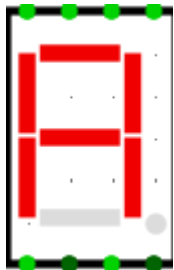
24 Hours counter

The 24 hours counter combines a modulo 10 and modulo 3 counter. Just using that configuration, however it would count to 29. In order to prevent this, we use an AND gate which resets both counters to 0 as soon as the tens place is 0010 and the ones place is 0100 or equivalently outputting 24. We also used a constant Low to convert the two-bit output of the modulo 3 counter to a 4 bit output.



12 Hours counter

The 12 hour counter works similarly to the 24 hour counter however it has a few extra complications. It uses a flipflop (Modulo 2 counter) together with a custom modulo 10 counter “For 12”. The subcircuit uses a D flipflop to start high and then output Low once simulation begins. This ensures that the clock begins on 12 AM using the reset function of For 12. After this the clock resets to 0000 and 0001 or equivalently 01 using the same style AND gate except it resets at 0001 and 0011 or equivalently 13. When the counter reaches 12 another AND gate triggers a T flipflop which controls the one segment difference between A and P in a seven-segment display. The AM/PM output is only active when the input SET12Hour is High.



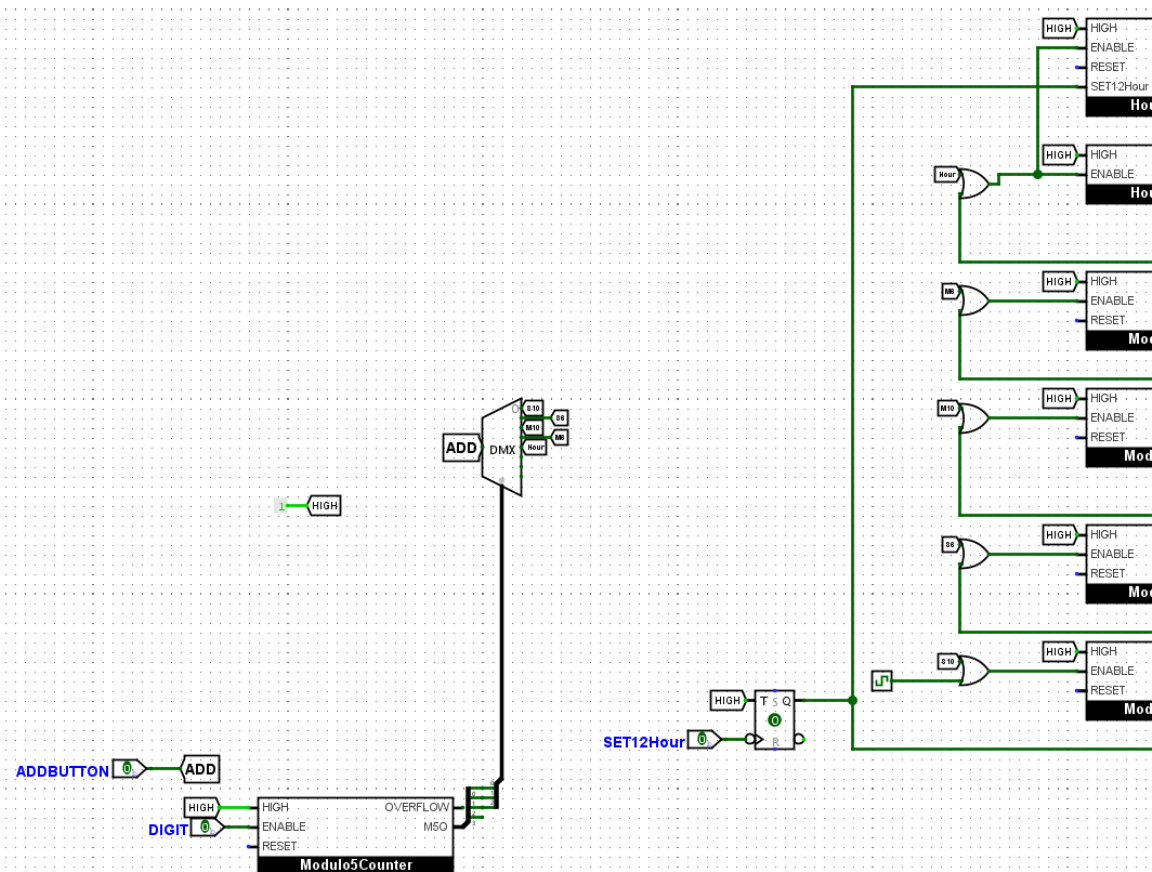
For 12

The custom modulo 10 counter “For 12” has inputs RESET which resets the output to 0010 (2) and SET which sets the output to 0001 (1). This is done through the use of the SET and RESET inputs of the T flipflops in the counter. The counter also resets to 0000 when it reaches 1010. Any overlap between these three was handled using OR gates for the input.

When SET12Hour triggers the T flipflop to output High it turns on the AMPM output and it changes the output of the multiplexers to the output of the 12 hour clock. To ensure that both clocks maintain an equivalent value, their enable inputs are fed simultaneously by the overflow of the minutes.

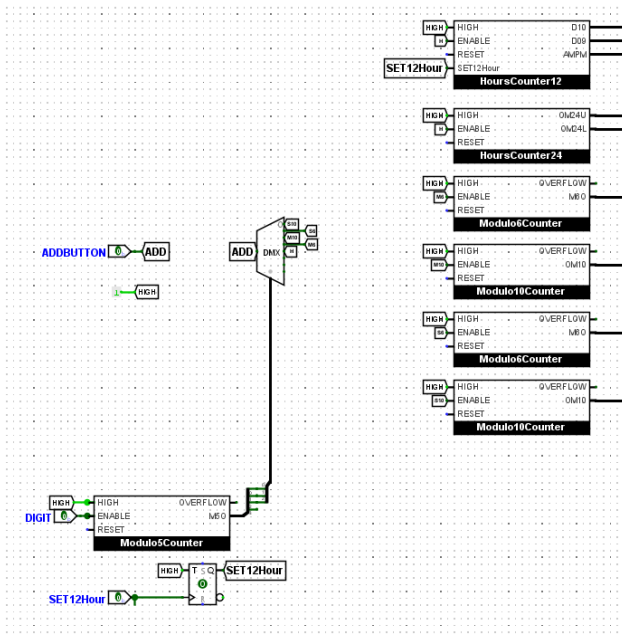
Setting time

In order to functionally set the time, we had two goals: To change digit and to add to the current digit. A modulo 5 counter was used in tandem with a 3-8 demultiplexer. This allowed for the user to switch through the units and tens place of seconds and minutes and the hours place (S1, S10, M1, M10, H). Due to the internal logic of our HoursCounter12/24 we could not use the tens place. We deemed this acceptable as there are only 24 numbers to cycle through. The modulo 5 counter was controlled by an input DIGIT which is linked to a button in the user interface. The input of the demultiplexer is an input ADDBUTTON which is connected to another button in the user interface. The outputs of the demultiplexer were then connected to the ENABLE input of each of the corresponding counters through an OR gate with the overflow of the previous counter or the clock input.



4. Alarm Mode

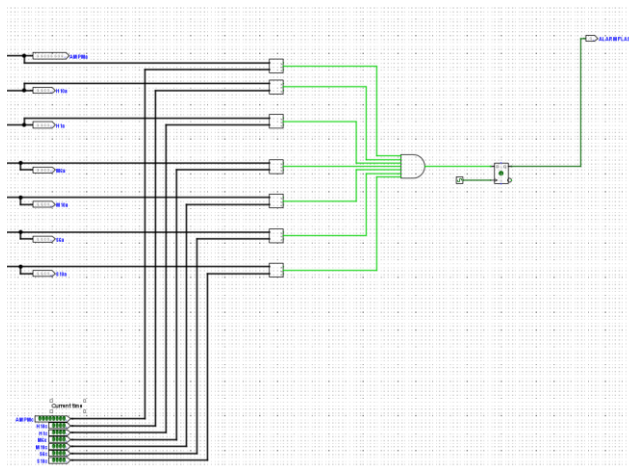
Setting Alarm Time



Setting the alarm is very similar to setting the time however we removed the overflow logic as this made things more user friendly in the case of an error and also removed several gates.

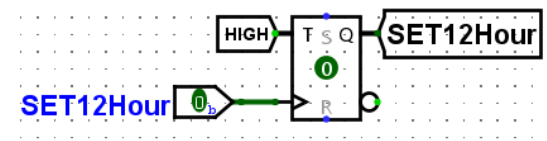
The outputs of the counters are displayed in the user interface when in the Alarm mode. This of course allows the user to see what time their alarm was set to.

Comparators

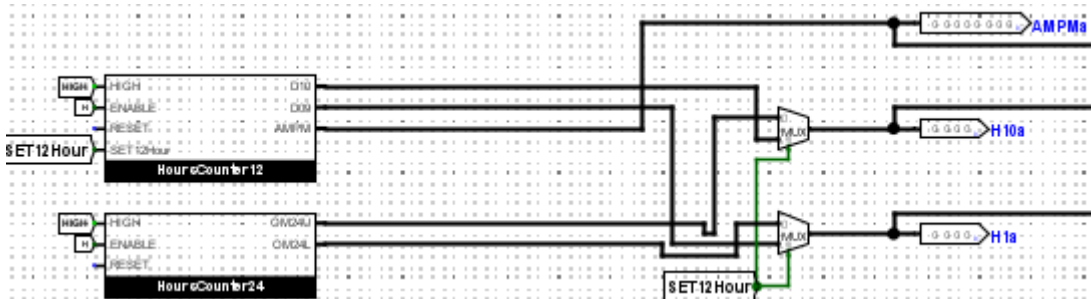


In order to trigger the ALARMFLASH we used a series of comparators which compared the output of the current time with the output of the set alarm. When the values of all digits and the AMPM were equal it would output High to trigger the alarm. We had an issue with the alarm being accidentally triggered due to a build-up of delays between different components. To solve this issue we ran this signal through a D flipflop synched to a clock signal. This ensured that the output ALARMFLASH was only triggered on this rising edge of the clock signal. This removed any irregularities caused by previous delays.

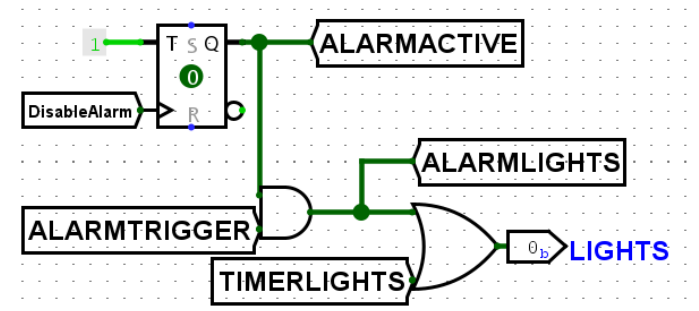
Switching Between AmPm



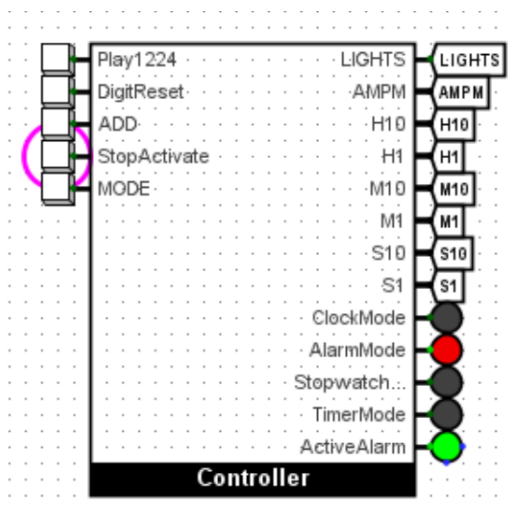
In order to switch between Am and Pm we used an input connected to a T flipflop. The output of said flipflop was used for two purposes. Firstly, it activates the AmPm output of the 12 hour counter, secondly it controls the multiplexer which controls whether the 24 hour is displayed or the 12 hour is displayed.



Enabling Alarm



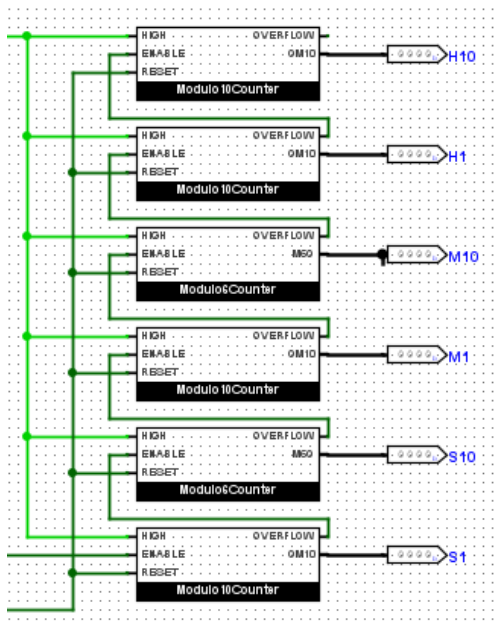
The alarm defaults to being disabled, when the DisableAlarm input is triggered by a user-controlled button it Enables/Disables. It prevents the activation of the alarm lights using an AND gate when it's Low. The output of ALARMACTIVE is shown to the user.



5. Stopwatch Mode

Stopwatch Count

The stopwatch was a comparatively simple design procedure as we had already created the functioning clock mode. Just like in the clock feature we needed to increment the display by 1 for every clock cycle. We used a combination MODULO 10 and MODULO 6.



As shown, the main difference from the incrementation of the stopwatch and clock design is that instead of going to 24 hours, we used 2 MODULO 10 counters for the hours so the stopwatch could reach a maximum of 99.59.59.

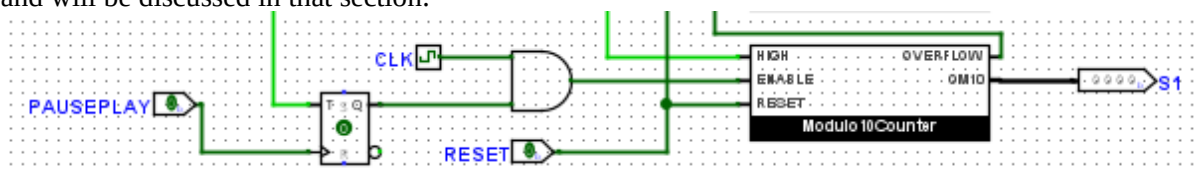
Stopwatch PausePlay

The stopwatch needed to include the feature to start it and stop it, we achieved this by connecting a button to the clock cycle of a T flip-flop. When the button was pressed it would enable the flip flop, this output would then go through an AND gate with the clock cycle. That would then enable the Stopwatch count in increments of the clock cycle. To pause the stopwatch the user can simply press the button again that switches off the T Flip-Flop.

Stopwatch Reset

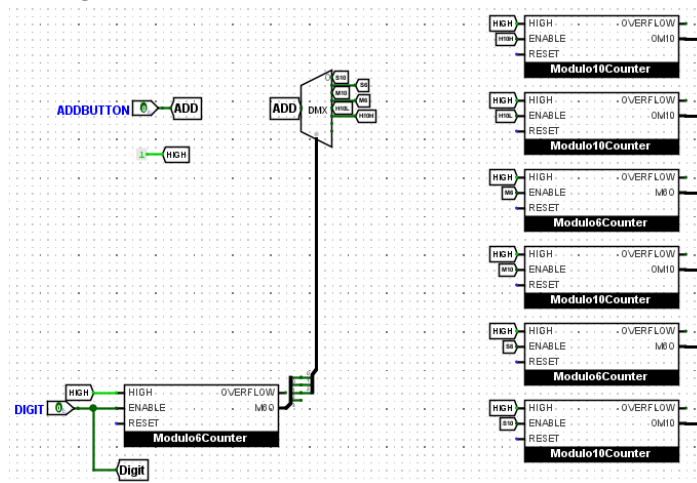
For the reset of the stopwatch a button that is linked in the controller to be triggered when the user presses the 'DigitReset' button. This connects to the reset input of all the MODULOS in the subcircuit, this resets the stopwatch to 00.00.00.

These two functionalities are shown below, the reset is connected to the Digit Reset in the controller and will be discussed in that section.



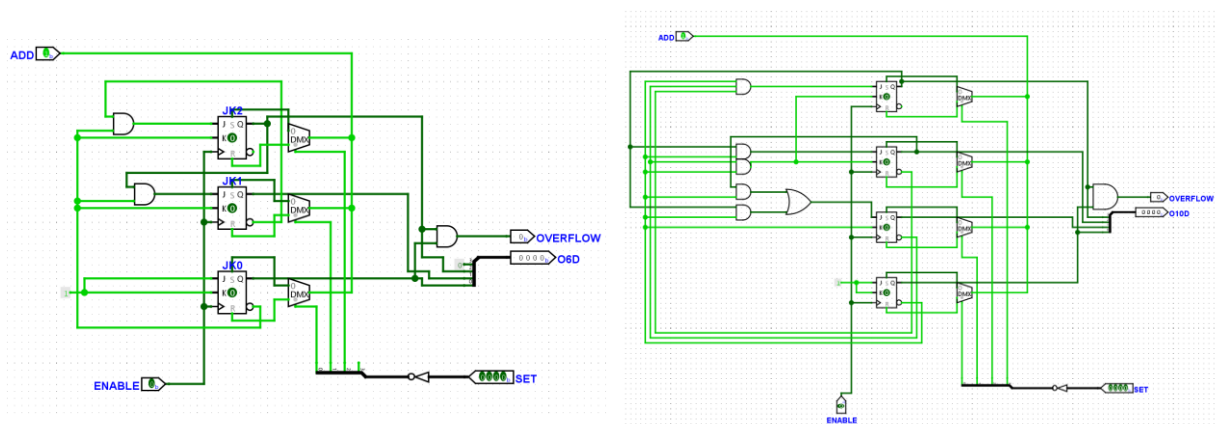
6. Timer Mode

Setting Time

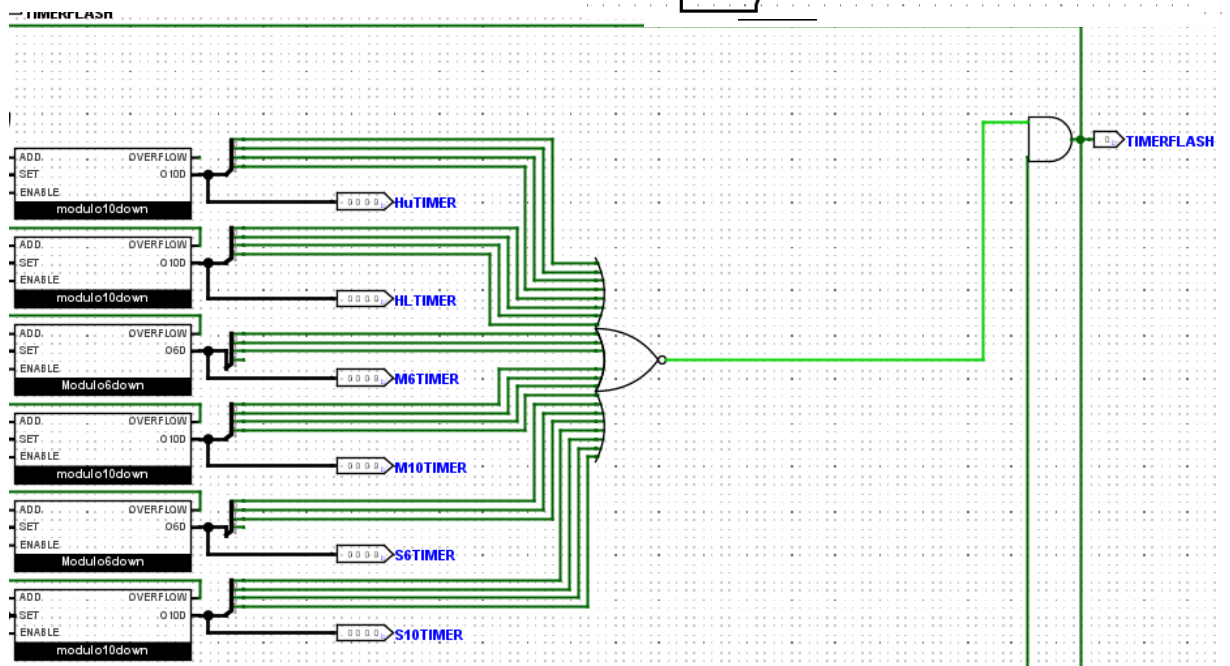
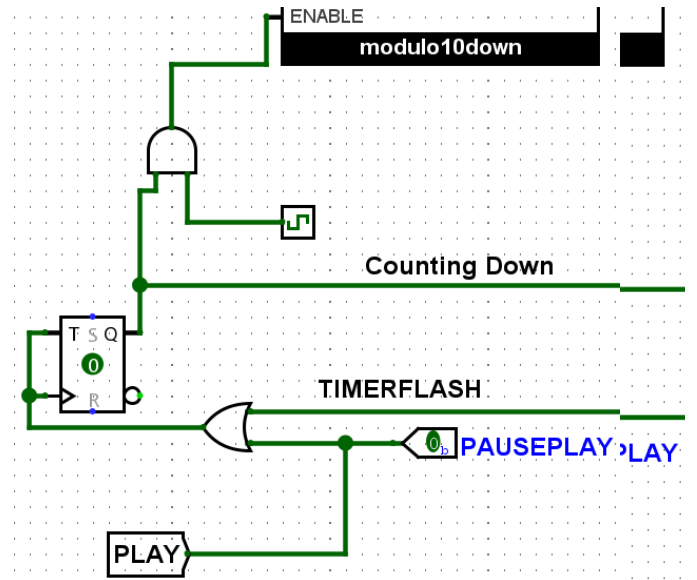
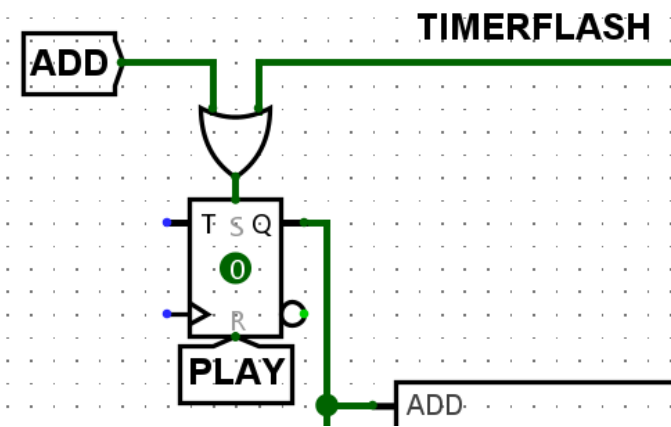


Controlling your timer is very similar to setting the time and setting your alarm, however the timer can be set up to 99:59:59. As the 12/24 hour counters have been replaced by Modulo 10 Counters we are able to control both digits of hours. In order to do this we have to use a modulo 6 counter to control the demultiplexer. The outputs are now (S1, S10, M1, M10, H1, H10). However these counters can only go up and are therefore used to set a series of “down” counters.

The output of each of the regular counters is inputted into a splitter in each corresponding down counter. These “Set” values are each used to control the select bit of an individual demultiplexer. One output is wired to the Set value of a flipflop while the other output is wired to the Reset value. Thus when the input of the DEMUX is high all the flipflops are set to the corresponding values found in the regular counters controlled by the user.

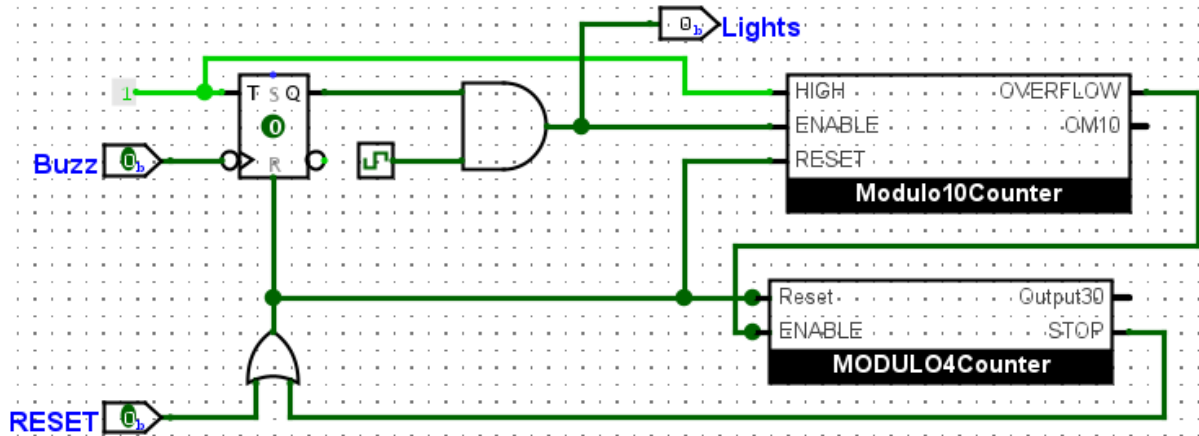


Several inputs are used to control the add function: ADD, TIMERFLASH and PLAY. When the user begins adding to their timer the output is set to constantly update, when TIMERFLASH is triggered, the output is reset and held to the previously set timer. Finally, if the user clicks play the down counters are released from the set value. This in conjunction with the now active AND gate allows them to count down to zero. The TIMERFLASH is triggered by a NOR gate when all inputs are 0 i.e. the end of the timer which will be further discussed below. TIMERFLASH also stops the clock from continuing to countdown. The user can also manually pause and play the clock similarly to the stopwatch using the PAUSEPLAY input



7. Flash

When buzz is triggered by the alarm or timer it makes the T flipflop output High. This begins the 30 second countdown and simultaneously flashes the alarm lights once per second. The countdown is paused and reset to 30 seconds in two ways. If the 30 seconds expires it will stop, or it can be manually stopped by the user through the reset input.

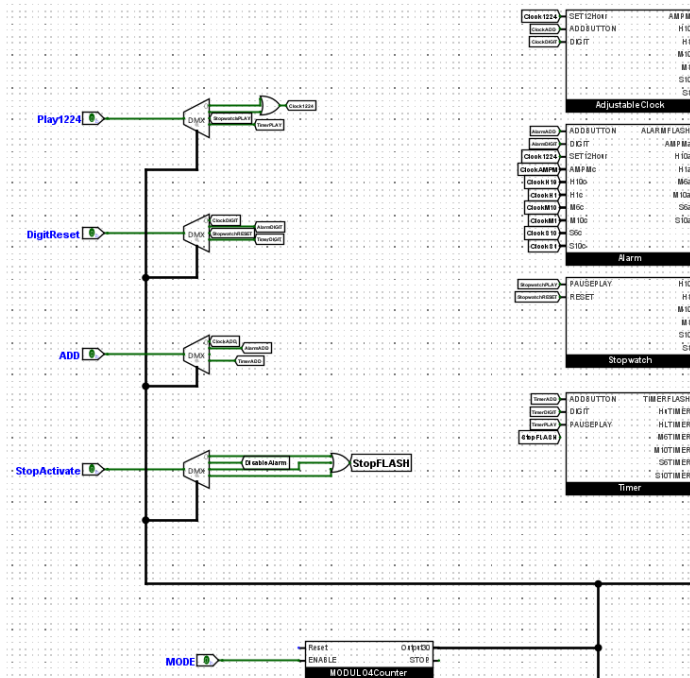


8. Controller

Switching Modes

In order to facilitate switching between modes we used a series of demultiplexers, and multiplexers controlled by a modulo-4 counter connected to the MODE button in the user interface. This counter would allow for the input passing demultiplexers and the output passing multiplexers to be moved in sync. The outputs of the demultiplexers are tunnelled (for clarity) into the corresponding inputs of each of our four main subcircuits (function discussed in subcircuits).

Inputs



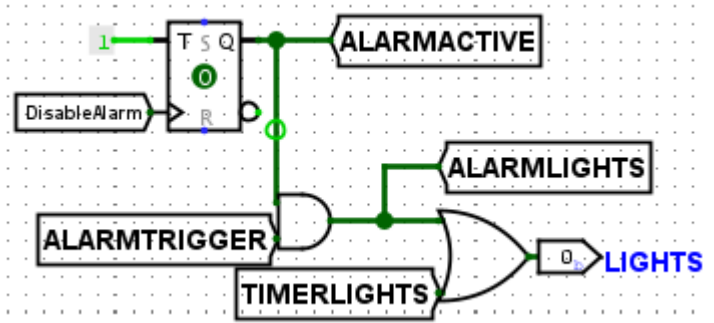
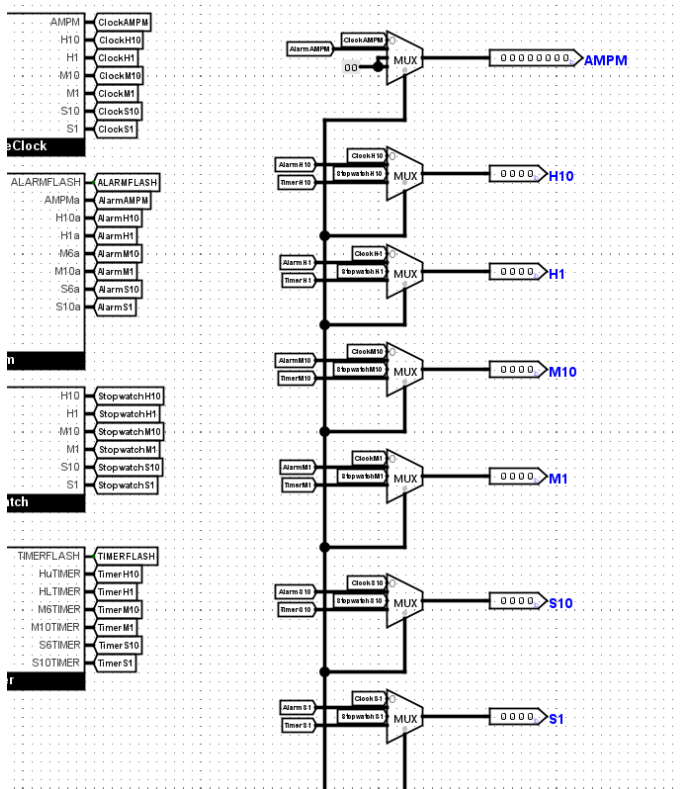
Each demultiplexer has an input from a user-controlled button. To maximise the simplicity and neatness of our user interface we tried to give buttons multiple functions to cut down on the number of demultiplexers and buttons needed. The dual functions could not overlap in any subcircuit which led us to our current design. The function of each of our buttons is explained in our User Manual.

Outputs

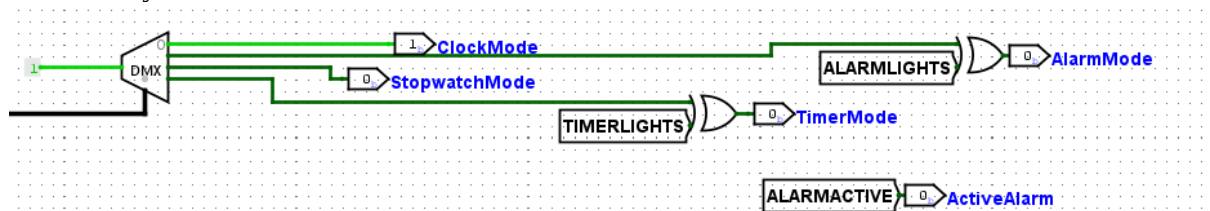
All of the circuits have outputs for each digit of the clock. Which mode the clock displays is controlled again by the counter and multiplexers. As well as the digits of the clock, the timer and alarm trigger flash outputs start the flashing of the lights. The clock and alarm can also display Am/Pm.

Disabling the alarm prevents the lights from flashing in and controls a green light which shows if the alarm is active or inactive. The T flipflop acts as switch which can be set by the disable alarm input button.

The LED outputs which indicate the current mode are also controlled by a demultiplexer and the modulo counter. To indicate which alarm has been



triggered the corresponding indicator light will flash. To maintain this function while on the same mode, we implemented an XOR gate which ensures it can still flash even when it should already be turned on by the mode indication.



9. User Interface

Clock Display

Controller has individual outputs for each digit of the clock and the AMPM display. The digits of the clock ran through BCD converters which connected to seven segment digital displays and converted the binary signals inputted in to the corresponding number. We had to manually connect up the AMPM display using splitters.

Mode

The controller also has a series of outputs to red LEDs. These are used to indicate which mode the clock is displaying and controlling. The AlarmMode and TimerMode also flash when the lights go off to indicate which setting has been triggered. As the Alarm can be disabled or enabled, there is also an output to indicate it is active which is a green LED to distinguish this output from the mode indicators.

Flashing LEDs

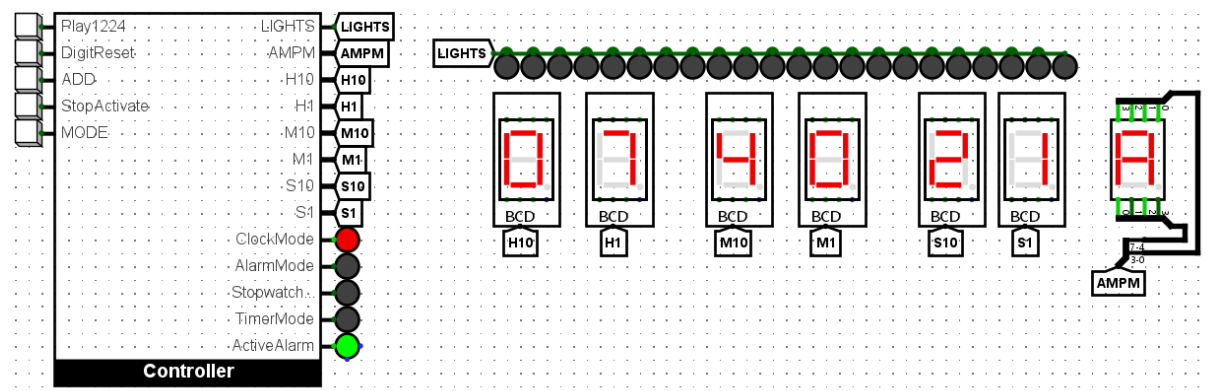
We indicate that the alarm or timer has been triggered through a line of several LEDs flashing for 30 seconds or until manually stopped by the user.

Buttons

We used buttons to control the clock rather than pins as they only need to be clicked once to trigger flipflops which was the majority of their functionality. The controller only uses five buttons to control and navigate all functions of the clock as outlined in our user manual.

Tunnelling

Tunnelling was used to present a more neat design in the display and allow for an easy understanding of the links between the outputs of the controller and which part of the display they interact with. We also used tunnelling in other subcircuits to reduce the amount of wiring and maintain clarity in our subcircuits.



10. Number Of Transistors

To begin this daunting task to count the total number of transistors that were used, we had to count the number of times each subcircuit was embedded into the entire clock and calculate the number of transistors in each subcircuit.

Using the User Interface statistics, we were able to simplify this task. By focusing on the total number of times each component was used, this was obtained from the recursive column.

After obtaining these results, we removed all components that do not use transistors

The results are shown below.

NOT Gate	6
AND Gate	79
OR Gate	54
NOR Gate	1
XOR Gate	2
Multiplexer	11
Demultiplexer	30
Comparator	7
D Flip-Flop	3
T Flip-Flop	121
J-K Flip-Flop	28

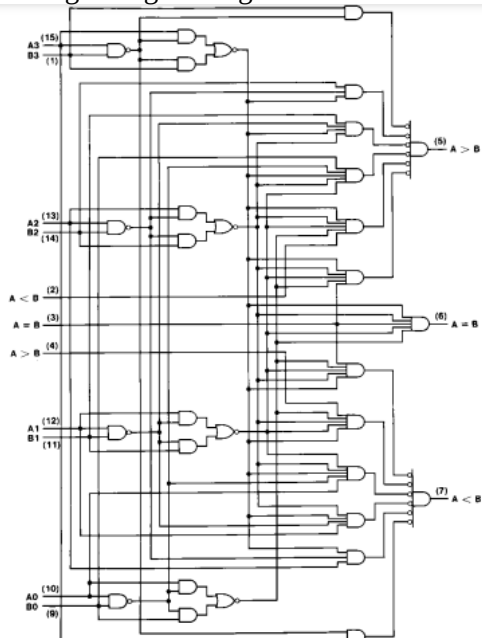
Unfortunately, from these elements the number of inputs are not specified, which would vary the number of transistors for some of the components. We needed to search through the circuits manually to find the number of each component with varying inputs.

To find the number of transistors in each component, we calculated the internal amount of logic gates needed for the Multiplexers, demultiplexers and comparators and from there could easily find the number of transistors in each component.

The comparators used in our design are Identity Comparators, however in Logism there was only an option of a magnitude counter so we used these instead and ignored the greater-to equalities.

To calculate the number of Transistors in both the 8-bit and 4-bit digital comparator, we modelled our count off the 'DM74LS85 4-Bit Magnitude Comparator'.[1]

The Logic diagram is given below



In our clock, as we would be using an Identity comparator, for our transistor count we removed the gates needed to calculate the greater-than and less-than inequalities.

This left us with four circuits, composed of 1 NAND, 1 NOR and 2 AND gates. Along with the four circuits we had to include a 4 Input AND gate to combine these circuits.

This left a total transistor count of 90 transistors for the 4 bit and 180 transistors for the 8 bit Comparator.

For the Flip Flops, using the circuit designs from sequential logic, we were able to determine that the number of transistors for each flip flop was,

$$D=2(D \text{ Latch})+NOT=36$$

$$T=D+XOR=40$$

$$JK=D+OR+2(AND)+NOT=56$$

$$XOR=2(AND)+OR+2(NOT)$$

For calculating the total number of transistors, we used a table in excel shown below,

	Count of Element	Components															
		Input Number	Demux				Mux		Comp		AND		OR		XOR		NOR
		Transistor Count	70	36	14	48	20	90	180	6	2	3	7	2	3	2	22
Controller	1			5		7				1				2		2	
Adjustable	1		1				2							5			
Alarm	1		1				2	1	6				1				
Stopwatch	1									1							
Timer	1		1							2				2			1
Flash	2									2							
Module10Counter	16									1				1			
Module6Counter	9									1				1			
Module5Counter	2									1				1			
Module4Counter	3									1							
Module3Counter	2									1				1			
module10down	4				4					5	1			1			
Module6down	2				3					3							
HourCounter2	2									1				0			
HourCounter1	2									2	1						
Far12	2									1				2	2		
TOTAL			3	5	16	7	4	1	6	74	6	1		84	4	2	1
Transistor Count	JK	D	T	NOT Gate													
Times used		56	36	40	2												
		28	3	121	6												

Using this table, we calculated the total number of transistors by multiplying each of the components transistor count by its number of times used.

Transistor Count=10204.

11. Conclusions

Summary

Using the detailed design procedure examined above, the digital clock that we have made using Logism successfully functions. The clock has a functioning alarm that flashes an LED bar when triggered. Along with successfully functioning timer and Stopwatch. The user can easily navigate the clock as only needs to use four dual functioning buttons plus a button to switch between modes. The user can switch between 12- and 24-hour modes and is clearly shown when in each mode.

In total we used 20 different Logism components, with a total component count of 982. Out of these components a total of 10204 transistors was used.

[Comparing these to estimations of the numbers of transistors in a Casio watch (from not so reliable sources), having counts between 20,000 and 30,000 transistors, we think it is time to mass produce our design!]

Potential improvements

As mentioned in the report above there were few improvements we could have made in the design. Our transistor count was quite high. A possible reduction could've been obtained with the substitution of the use of JK flip flops, initially we did not know how many more transistors were used in JK flip flops as opposed to T Flip Flops, so could've created more efficient modules.

Conduction of Co-Design

We both worked together for all components of the clock design. This included discussing possible solutions to each task at hand. As we were unable to work simultaneously on the logism circuit, this was in majority done together. We split up the formation of counters but the more complex layout of the logic in the clock was executed together. Both of us conducted multiple rounds troubleshooting of the clock separately.

For the report and User manual we split up the workload, taking an even amount of headings. We proofread the entire report multiple times together and changed any errors accordingly.

Sources;

[1] CD4063BMS Datasheet <https://www.futurlec.com/Datasheet/CD4000/CD4063.pdf>